

Assignment 1, Task 1,2 Report

Muhammad Ahmad · 25280065 · LUMS, Fall 2026

<https://github.com/Ahmad-Gill/LUMS-Academics/tree/main/Third%20Semester/Deep%20Learning%20For%20Time%20Space%20Grsph/Assignments/Assignment%201>

Summary of results (validation split)

Model	Established RMSE	Station 4 RMSE	Parameters	Fit time
Shared ridge	0.767	0.876	4,608	0.001 s
Sensor-specific ridge	0.768	unavailable	13,824	0.001 s
Period-routed ridge	0.166	0.173	59,904	0.002 s
Raw Attention	0.424	0.617	368,368	441 s
Attention + decomposition	0.287	0.428	373,024	519 s
Raw delay mixer	0.219	0.337	368,370	487 s
Autoformer-inspired	0.199	0.240	373,026	560 s

Response 1A: Predict, measure, and revise the aggregate ordering

(a) Expected ordering, written before Output 1.1: period-routed ridge best, then Raw Attention, then shared and sensor-specific ridge about equal and worst. Same order for station 4, where sensor-specific ridge is unavailable.

(b) Comparison across populations and products (Output 1.1, validation RMSE):

Model	Est. A	Est. B	Est. C	Stn 4 A	Stn 4 B	Stn 4 C
Shared ridge	0.805	0.854	0.621	0.909	0.982	0.713
Sensor-specific ridge	0.804	0.852	0.630	n/a	n/a	n/a
Period-routed ridge	0.181	0.164	0.152	0.184	0.182	0.150
Raw Attention	0.387	0.476	0.405	0.649	0.578	0.621

(c) Key differences: routing beats Raw Attention by 0.26 on the established stations and 0.44 on station 4. Sensor-specific ridge gives no gain over shared ridge (0.768 vs 0.767). Raw Attention loses 0.19 when moving to station 4, while routing loses only 0.007.

(d) Revised ordering: the predicted order held, but the gaps were much larger than expected. Sensor-specific ridge is no better than shared ridge, and Raw Attention generalises poorly to the unseen station.

(e) Explanation: shared ridge assumes one map fits every pace, so it compromises. Sensor-specific ridge assumes windows differ by station, but the pace changes inside every station. Period routing conditions on the pace itself, so it stays accurate on every product and transfers to station 4. Raw Attention adapts per window but must learn around the large drift, and it partly fits station-specific patterns.

Response 1B: Why the forecasting rule must depend on context

(a) Output 1.2 (Figure 1): the same station under Product A and Product C needs different phase advances. One fixed matrix $\hat{W}$ has no variable that selects the pace, so least squares averages the two. The shared ridge forecast comes out with reduced amplitude and its peaks move out of phase with the truth as the horizon grows.

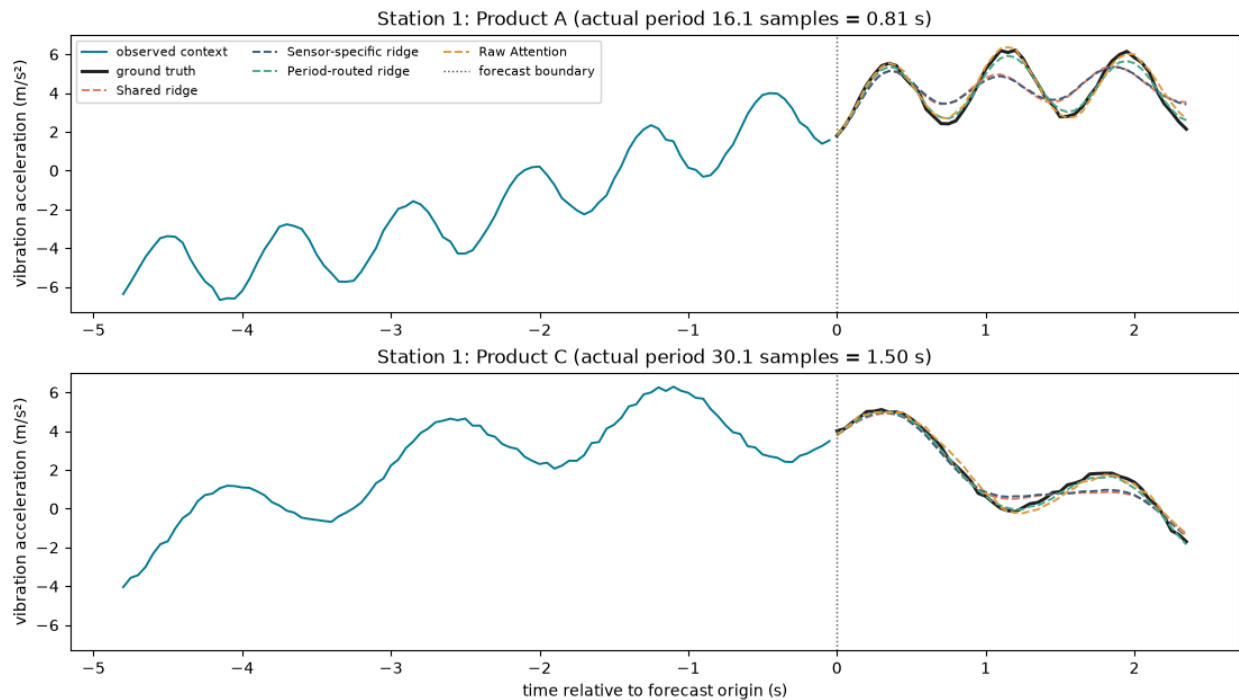


Figure 1. Output 1.2. Station 1 under Product A and Product C: forecasts of the four Part 1 models against the ground truth (validation).

(b) Output 1.3 (Figure 2): W_Q , W_K and W_V are fixed after training, but the weights $\text{softmax}(QK^T/\sqrt{d_h})$ are recomputed from each window. The Product A and Product C attention maps differ, and their difference panel is non-zero, so the same projections build a different value-mixing operation for each context.

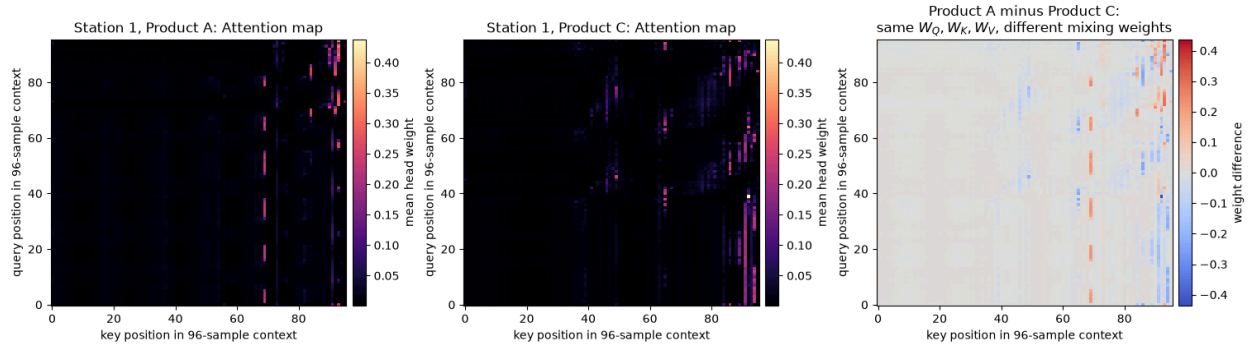


Figure 2. Output 1.3. Raw Attention maps for two contexts of the same station, and their difference.

(c) Contrast: ridge applies the same $\hat{W}$ to every window, while Attention builds a new mixing for every window. Output 1.3 only shows that the mixing depends on the input; it is not evidence that Attention has recovered the physical period.

Response 2: Separate slow structure from repetition

(a) Selected width and observed changes:

- **Selected width: 41.**
- Output 2.1 (Figure 3): a small width makes the trend follow the vibration (residual std 0.58 at width 17), while larger widths give a smooth trend and a full-amplitude residual (1.57 at width 41).
- Output 2.2 (Figure 4, training split): period recovery within one sample rises from 0.687 on the raw context to 0.998 at width 41.
- Output 2.3 (validation): RMSE falls from 0.424 to 0.287 on the established stations and from 0.617 to 0.428 on station 4, with gains on every product (for example, Product A established 0.387 to 0.261).

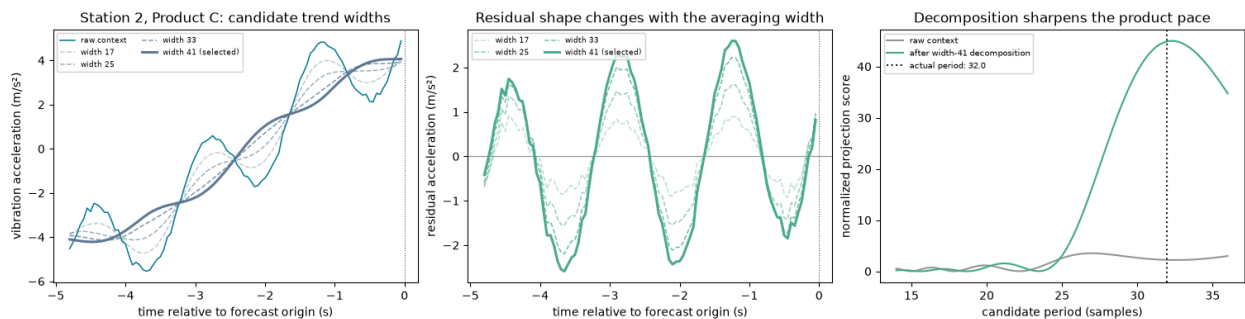


Figure 3. Output 2.1. Candidate trend widths, the resulting residuals, and period scores before and after decomposition.

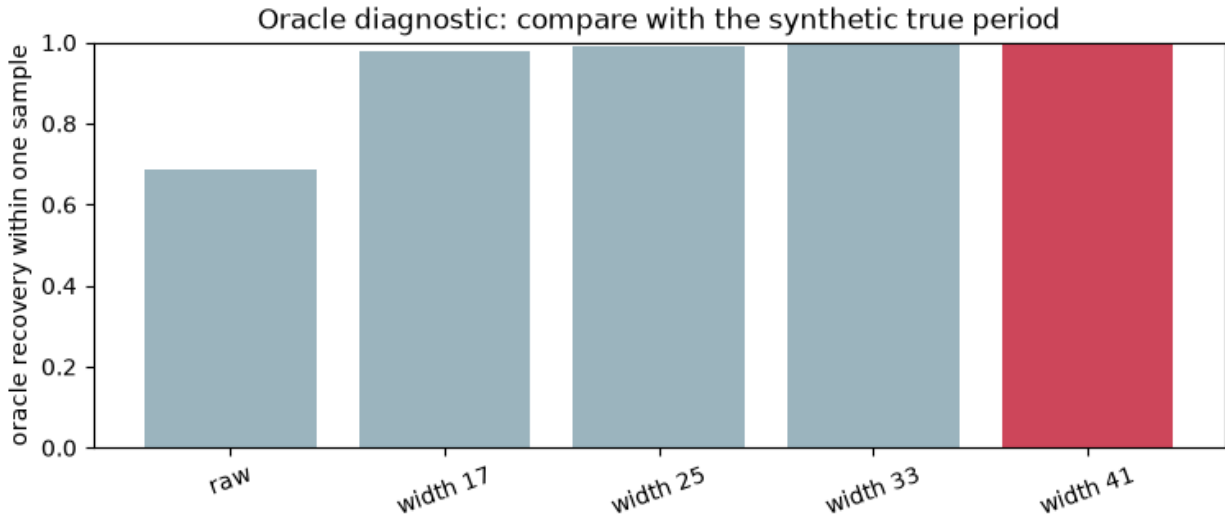


Figure 4. Output 2.2. Oracle period recovery for the raw context and each candidate width (training split); width 41 selected.

(b) Why this bias makes sense: the world is a slow drift (period 240 samples) plus a fast vibration (period 14–36). Over a 41-sample window the vibration averages out while the drift barely changes, so a moving average separates the two cleanly. The oracle diagnostic uses only training windows and the generator's true period, purely to choose the width; the period is never a model input, so the choice does not leak validation or test information.

(c) Where it would fail: if the slow component changed abruptly (a step change or a sudden fault), the moving average would smear it into the residual. An alternative is STL or a learned trend filter, which assumes a stable seasonal shape rather than a locally smooth trend.

Response 3: Mix by temporal delay

(a) Verification: Output 3.1 matches the direct correlation $[4, -4, 4, -4]$, and the aggregation example gives $z = [35, 15, 25, 25]$, confirming that position t reads from $t - \text{delay}$.

(b) Strongest lags: in Output 3.2 (Figure 5; station 2, Product A, actual period $P = 16.1$), the residual correlation peaks at lag 16 (0.90) and lag 32 (0.90), which are P and $2P$.

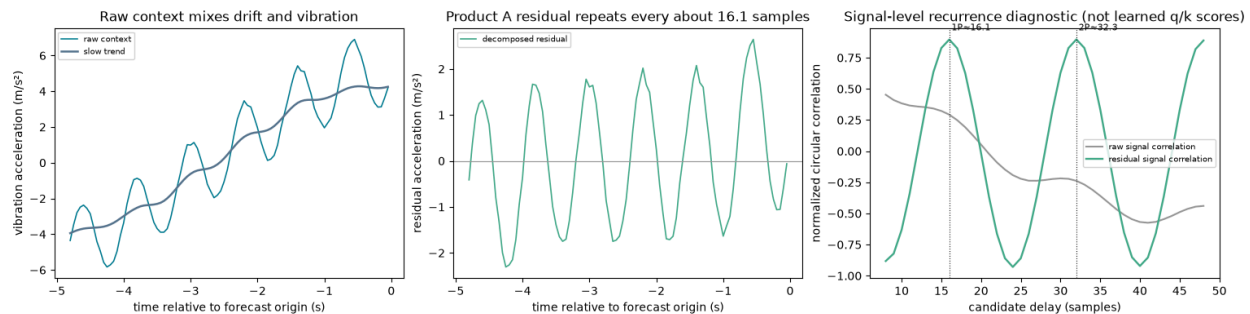


Figure 5. Output 3.2. Raw context and trend, the decomposed residual, and the signal-level correlation by delay (validation). This is a signal diagnostic, not the model's learned q/k scores.

(c) Why decomposition cleans the evidence: the raw context contains a large drift ramp that correlates with itself at almost every shift, which flattens and blurs the curve. Removing it leaves the vibration, whose correlation peaks sharply at multiples of P . This supports the delay mixer's assumption that useful relationships sit at a few recurring displacements.

(d) Where the assumption fails: an isolated transient such as a sudden jam appears once, so no delay predicts it. Mixing shifted copies would smear the spike across the forecast or copy it forward as if it repeated. A period that changes quickly inside one window also breaks the assumption, because no single delay lines up across the whole context.

Response 4: Compare designs and deploy

(a) Prediction, written before Output 4.1: decomposition should help most when the slow component is prominent, because it removes the drift ramp before mixing, so the mixer sees the repeating vibration.

(b) Validation results on the established stations (Output 4.1):

Model	RMSE (m/s ²)	MSE (m/s ²) ²
Shared ridge	0.767	0.588
Sensor-specific ridge	0.768	0.590
Period-routed ridge	0.166	0.028
Raw Attention	0.424	0.180
Attention + decomposition	0.287	0.082
Raw delay mixer	0.219	0.048
Autoformer-inspired	0.199	0.040

- Decomposition helps with both mixers: $0.424 \rightarrow 0.287$ with Attention, and $0.219 \rightarrow 0.199$ with delay mixing.
- Delay mixing helps in both settings: $0.424 \rightarrow 0.219$ without decomposition, and $0.287 \rightarrow 0.199$ with it.
- Combining both gives the best neural model.

Across the horizon (Output 4.2, Figure 6), every model starts near 0.11–0.15 at step 1 and its error grows with the forecast step as phase errors build up. On the established stations at step 48, Raw Attention reaches about 0.53 while period-routed ridge stays near 0.23 and Autoformer-inspired near 0.24. On station 4 the gap widens: Raw Attention rises to about 0.8 and the raw delay mixer climbs sharply after step 30, while period-routed ridge and Autoformer-inspired stay lowest.

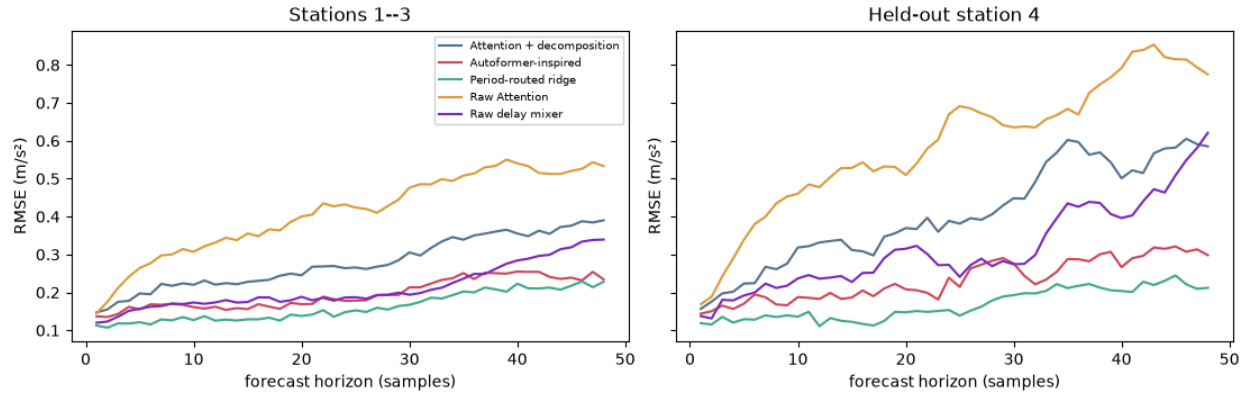


Figure 6. Output 4.2. Validation RMSE along the 48-step horizon for stations 1–3 and held-out station 4.

(c) Deployment choice:

- **Established stations: period-routed ridge.** Lowest validation error (0.166 RMSE, 0.028 MSE), 59,904 parameters against about 373,000, and fits in 0.002 s against 441–560 s.
- **Station 4: period-routed ridge.** Lowest validation error (0.173 RMSE, 0.030 MSE) against 0.240 (0.058) for Autoformer-inspired; it barely degrades on the unseen station, and sensor-specific ridge is unavailable there.

Test results (Output 4.3) confirm both choices: **0.161** for the established stations and **0.183** for station 4.

Task 2: Leaderboard Challenge

What the data looks like

The series has 43,656 values. Most are small but there are big spikes (0 to 994, average 98). It remembers the recent past but forgets quickly: the value now is strongly linked to the next hour (0.97), less to the same hour tomorrow (0.40), and not at all to the same hour next week (0.03). There is a weak daily cycle of 24 steps.

How I prepared it

- I scaled the target. I tried it both as it is (raw) and with a log transform.
- I scaled the external features using training data only, and used log on the very skewed ones (D, E, F).
- I added daily (24) and weekly (168) cycle features from the time index, because no dates are given.

The model

An Autoformer with one encoder layer and one decoder layer (width 32). It reads the last 168 steps and predicts the next 168. It uses my Question 1 code for both Autoformer parts:

- series decomposition (moving average, kernel 25)
- Auto-Correlation (delay scores with FFT, plus my delay aggregation)

The past external features go into the encoder. The **future** external features go into the decoder, because the decoder makes the future prediction. The design follows Wu et al. (2021) and github.com/thuml/Autoformer.

How I tested it

I kept the last 10 weeks as validation and made one 168-step forecast starting from each day (60 forecasts). I ran every model with 3 seeds and report the average.

Results (validation RMSE)

Model	RMSE
Training average (baseline)	109.42
Last value repeated (baseline)	157.13
Autoformer, log target	84.59 ± 3.52

Autoformer, log target, 336 input	85.81 ± 2.86
Autoformer, raw target	78.75 ± 7.87
Autoformer, raw target, lower learning rate	82.27 ± 6.03
Autoformer, raw target, 336 input, lower learning rate	82.05 ± 4.97
Final: average of 5 raw + 3 log models (0.6 / 0.4)	72.40

The raw target gave the best RMSE, because it predicts the average, which RMSE rewards. The log target gave a better MAE (52.6 vs 61.6). Single models changed a lot between seeds (73.7 to 87.8), so averaging several models gave a more stable and better result.

Did the external file help? (raw target, 3 seeds)

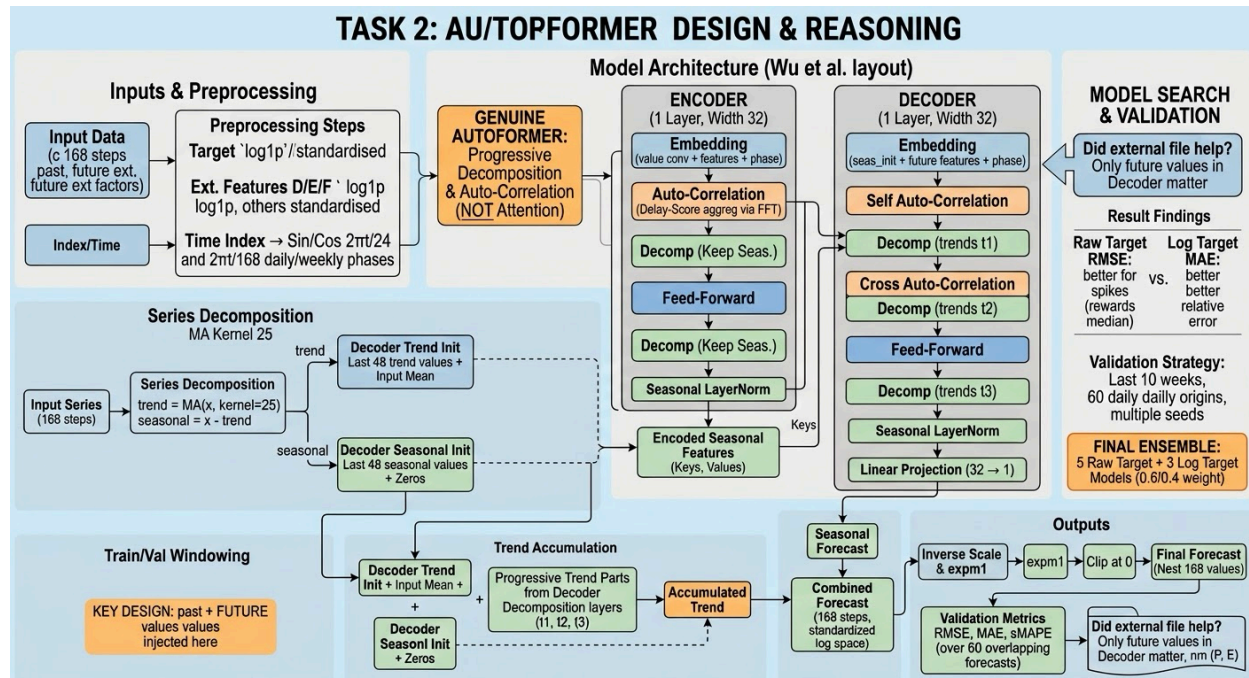
Input	RMSE
No external file	114.07
External file, past only	114.53
External file, past + future	78.75

Only the future values help. Past values add nothing, because the target already shows what happened in the past.

Leaderboard

- Attempt 1 (one model, P = 22,209, E = 8): RMSE 91.25
- Final attempt (ensemble, P = 177,672, E = 65): RMSE 87.4206

The size and epoch penalties turned out to be very small, so accuracy matters most. One week of test data is noisy, so I chose models using my validation results, not the leaderboard.



Q2.1

Auto-Correlation works better for signals that repeat with a clear, fixed period, like the vibration in Task 1. There the whole cycle repeats, so matching the series with a shifted copy of itself finds the period straight away, and every position uses the same few delays. In Task 1 the delay mixer reached 0.219 RMSE against 0.424 for Raw Attention on the established stations.

Auto-Correlation loses information when the important events do not repeat, such as one sudden spike or a local pattern that happens only once. It keeps only a few whole-series delays, so a single unusual moment is averaged away, while attention could still link that one position to a similar one elsewhere.

When the period drifts, Auto-Correlation suffers more, because one fixed delay no longer lines up across the whole window and the correlation peak gets smeared. Attention can still match points one by one, but it also needs more data to learn this. In our Task 2 series the periodic signal is weak (autocorrelation 0.40 at lag 24 and almost 0 at lag 168), so most of our accuracy came from the external features, not from periodicity.

Q2.2

A moving average assumes the trend is smooth and close to straight inside the kernel (25 steps for us). At the window edges there are not enough neighbours, so we repeat the edge value. This flattens the trend near the ends, which is exactly where the forecast starts. Where the trend curves, the average cuts off peaks and fills in valleys, so some curvature leaks into the seasonal part. An abrupt level jump gets spread over about 25 steps, so it appears as a slow ramp in the trend and as a fake wave in the seasonal part.

An alternative is a robust decomposition such as STL, or a learned trend layer. STL assumes a

stable seasonal shape and is robust to outliers, which helps with our spiky target. You give up simplicity and speed, and STL is harder to put inside the network at every layer.

Q2.3

Decomposing inside every block means any slow pattern the network creates while mixing is removed again before the next layer. The seasonal path stays clean, and the trend is built up step by step in the decoder. One-shot decomposition cannot do this, because it only cleans the raw input.

It can hurt when the trend part carries real information that has to be mixed with the seasonal part, for example a slow rise in level that changes how big the daily swing is. Repeated smoothing throws away the slow part at every step, so the network cannot learn that link. It can also hurt when the input window is short compared with the kernel, because then most of the window is edge effect.

Q2.4

k is how many delays are mixed. With $L = 168$ and factor 1 we used $k = 5$. A larger k includes more delays, which is safer if the signal has several periods but also adds weak or noisy delays that blur the output. A smaller k is sharper and cheaper, but can miss a real period.

The $\log L$ form is mostly convenient. It grows slowly, keeps the cost at $L \log L$, and gives a reasonable number for usual window sizes, but it does not come from the signal itself. Ideally k should match the number of real periods and their multiples in the data. Our series has a weak daily cycle, so a small k was enough. We did not test other k values.

Q2.5

One-shot (what Autoformer does, all 168 steps at once): errors do not feed into each other, and the model is trained on exactly the task it does at test time, so there is no exposure bias. The errors across the horizon tend to be smooth and correlated, because one hidden state produces all of them.

Feeding predictions back (autoregressive): each step uses earlier guesses, so mistakes pile up, and the model is trained on true inputs but tested on its own guesses (exposure bias). It can be better for short horizons with smooth, stable dynamics, where one step ahead is easy.

For long horizons like our 168 steps, one-shot is usually safer. Our series loses most of its memory after a day or two (autocorrelation 0.97 at lag 1, 0.16 at lag 48), so a recursive model would mostly be feeding its own errors back in.

Q2.6

Window normalisation removes the window's average level and spread. What is lost is the absolute level: whether this window is high or low compared with the long-term normal. That matters when the level itself predicts the future, for example when high values tend to fall back towards the average.

It helps a lot for series whose level drifts over time while the shape stays the same, such as sales that slowly grow. It loses something essential for a series like ours: a heavy-tailed, non-negative quantity whose spikes fall back to a typical level. After window normalisation a calm week and a polluted week look alike, and the model cannot know which way to move. We used global standardisation, with statistics from the training data only, to keep the absolute level.

Q2.7

Yes, it changes where they should go. Features known only up to the forecast origin can only enter the encoder. Features known across the horizon should also enter the decoder, next to the steps being predicted.

Our ablation (validation RMSE, mean over 3 seeds) shows this clearly:

- no external file: 114.07
- external file in the encoder only (past values): 114.53
- external file in the encoder and decoder (past and future values): 78.75

A covariate like next week's wind speed is very useful when it is known for the horizon, but almost useless as a past value only, because the target already reflects past conditions. A covariate that is equally useful in both cases is the time-of-day phase, which is fully predictable, so knowing its past values tells you its future values.

Q2.8

Without dates, the model cannot use calendar effects directly: hour of day, day of week, holidays or seasons. We recovered the period from the data instead. The autocorrelation and the periodogram showed a clear 24-step cycle, and the average value by step mod 24 showed a daily pattern (from about 86 to 113). We added sine and cosine features for periods 24 and 168.

For this to be trustworthy we have to assume the sampling is regular (the handout guarantees this), that the period stays the same over the whole series, and that the pattern is strong enough to stand out from noise. Here the weekly pattern was weak (daily means by weekday 92 to 103), so we could not be sure it is real.

Q2.9

Structural models win when the series is short, the noise is high, and the pattern is regular, because their strong assumptions do the work that data cannot. A decomposition Transformer needs a long series and many training windows, and it pays off when the patterns are irregular or depend on outside variables.

Our series is long (43,656 points), noisy and heavy-tailed, with weak periodicity, and it has useful external features. That puts it on the Transformer's side. Before training, I would check the series length, the strength of the autocorrelation at the seasonal lags, and simple baselines. In our case:

- training mean: 109.42
- persistence: 157.13
- linear model with future features: about 90
- Autoformer: 78.75 (single model), 72.40 (blend)

Use of AI

I used Claude (Anthropic) strictly for detailed explanations and to gain a better visual understanding of the assignment.